

Sobrecarga ← Métodos con el mismo nombre pero **DIFFERENTES** parámetros (número de parámetros, tipo u orden)
↑
enlace estático
Resuelve en compilación

Sobreescritura ← Redefinir un método heredado del padre en la clase hija, manteniendo la misma firma.
↑
enlace dinámico
Resuelve en ejecución

Herencia ← Mecanismo que permite que una clase herede atributos y métodos de otra clase.

Por extensión (extends)

- **NO** tiene acceso a los atributos y métodos privados
- **NO** puede cambiar la naturaleza de los públicos/protected del padre.
- + Puede crear un atributo con el mismo nombre que un público, pero entonces, lo sobreescribe en la clase (tendrá el propio y el del padre).
- + Puede sobrecargar los métodos del padre (y sobreescribirlos)
- + Los hijos tienen la naturaleza de los padres como parte de la herencia de atributos, métodos y relaciones (polimorfismo).

Lo que puedo llamar { Tipo de la referencia
Lo que se ejecuta { Decide } Clase real del objeto

↑ Relación del sistema de tipos, de manera que una referencia a una clase (atributo, parámetro, declaración local, ...) acepta direcciones de objetos de dicha clase y de sus clases derivadas (hijos, nietos...).

Por implementación (implements) ← **Interface**

Interfaces {
+ Constantes (static final)
+ Métodos abstractos
+ Método static
X Nada de constructores ni instancias.

↑
Con la incorporación del polimorfismo, tiene sentido declarar referencias a clases abstractas con la intención de almacenar direcciones de objetos de clases concretas derivadas, NO de clases abstractas que no son instanciables.

• Son **clases abstractas puras** y por tanto **nunca instanciables**.

• Todos los **métodos** y **atributos** de la interfaz son **públicos**.

- Todos los atributos definidos en una interfaz serán siempre **public static final**, aunque no estén definidos los modificadores.

- Todos los métodos de la interfaz son **públicos**, aunque no estén definidos los modificadores.
Solo se puede incluir cuerpo en los métodos static.

Clase abstracta ← Son **clases NO instanciables** que surgen del factor común del código de otras clases con **atributos comunes, métodos comunes**.

Método abstracto $\Rightarrow$ Clase abstracta
 $\nwarrow$ ~~abstracta~~

Una **clase abstracta** puede tener sus propios métodos abstractos y concretos, y usar métodos de otras clases.

Un **método concreto** puede usar **métodos abstractos** de la misma clase.

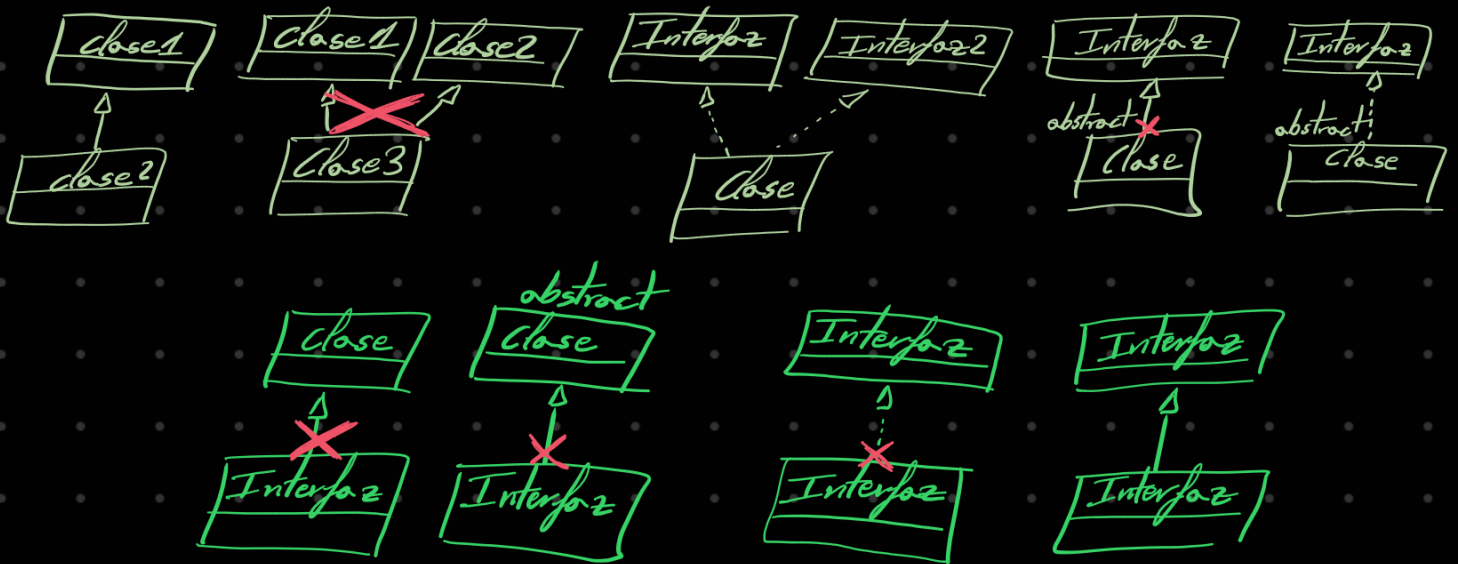
La **clase hija** debe implementar los **métodos abstractos**.

sobreescribe

El **atributo super** en las clases derivadas hace **referencia al padre (superclase)**, tanto en **métodos** como en **atributos**.

Todas las clases en Java, directa o indirectamente, heredan de **Object**, lo que por herencia tienen métodos como **equals**, **toString**, **hashCode**, **clone** y **finalize** que puedes usar o sobreescribir.

Extensión Vs Implementación (En Java)



La herencia por extensión NO disfruta de herencia múltiple, pero la herencia por implementación SI disfruta de herencia múltiple.

Las clases implementan interfaces.

Las interfaces expanden otras interfaces.

Final

- Clases final: no permiten ningún tipo de herencia posterior.
- Métodos final: no permiten ningún tipo de redefinición posterior.
- Objeto final: no se pueden volver a instanciar (nuevo new) ^{← Pero sí modificar su contenido.}
- Los enumerados son siempre final, No pueden heredar de una clase por extensión, pero SI puede heredar de un interfaz por implementación.

Static

- NO pueden ser sobrescritos (hiding).
- NO hay polimorfismo (aunque se intente permanece lo estático (compilado)).
- Solo puede acceder a instancias estáticas.
- + Pueden sobrecargarse.

El método estático pertenece a la CLASE, no a un objeto en específico.

Constructores y orden de inicialización

No llamar a métodos sobrescribibles desde constructores porque los hijos no se inicializan en él y puede perder sentido la ejecución.

Conversión de Variables Dinámicas a Objetos

• **Conversión ascendente (upcast):** se transforma una dirección de un objeto de una clase a una dirección del objeto pero de una clase ascendente (padre, abuelo, ...).

Animal a = new Perro();

• **Conversión descendente (downcast):** cuando se transforma una dirección de un objeto de una clase a una dirección del objeto pero de una clase derivada (hijo, nieto, ...).

Perro p = (Perro) a;

• Cualquier otra conversión produce error en tiempo de ejecución:

conversión de una dirección de un objeto de una clase fuera de la rama ascendente o del árbol descendente, tíos, hermanos, clases ajenas...

Abstract no se puede instanciar

Un método abstract obliga que la clase sea abstract
pero una clase abstract puede tener métodos concretos

Un método abstract debe ser
implementado por el hijo que
lo hereda obligatoriamente

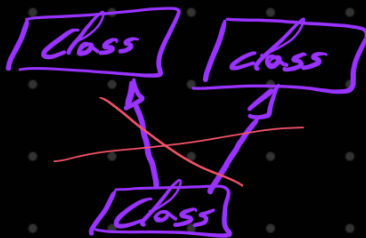
↑
o no ser que sean abstract
la única forma de acceder
a ellos es por un hijo que herede

Un atributo final debe asignarse valor en:

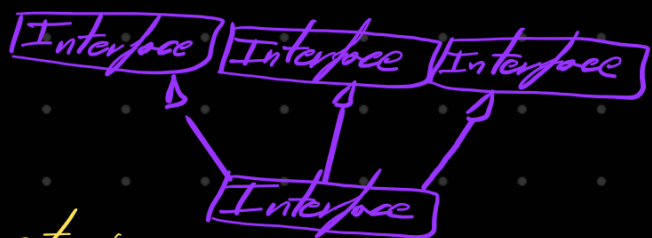
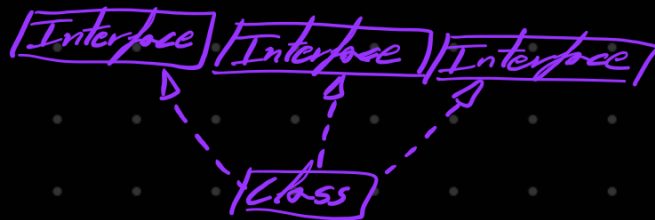
- la declaración

- el constructor

en caso contrario error de compilación



no se puede hacer
en Java, la herencia
es ambigua



Single Responsibility

Open/closed

Liskov substitution

Interface segregation

Dependency Inversion

Abierta a extensión
cerrada a modificación

Reduce acoplamiento

Uso de abstracciones para
no conocer las implementaciones
por dentro

Animal p = new Perro();
hijo sustituye
al padre

Control de clases

Inversion of Control (IoC) → En vez de decidir un empleado

- Es un empleado que
- solo sabe hacer su tarea
 - un jefe decide:
 - cuando trabaja
 - que tarea ejecuta
 - el orden

- que hacer
- cuando trabajar
- como trabajar

Dependency Injection (DI) → ¿Quién crea y proporciona los objetos que necesito?

↑
Forma concreta de aplicar IoC

↑
Una clase externa se encarga de crear el objeto e inyectarlo:

- Por constructor
- Por setter
- Por campo

Patrones de diseño

Creacionales { Singleton
Builder

Estructurales { Decorator

Comportamiento { Command
Observer

Genéricos <?>

Excepciones

Aserciones

Comportamiento (que no sobrescritura)

```
public class Parent() {  
    public static talk() {  
        system.out.println("parent");  
    }  
}  
  
public class Child extends Parent() {  
    public static talk() {  
        system.out.println("child");  
    }  
}
```

Parent.talk(); → parent
Child.talk(); → child